SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: *Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL: 3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Design Critique Session

Code of Conduct:

I Pusalapati Chandu(192372119) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary

action. ***Signature of the student:*** 

SIMATS ENGINEERING
ASSESSMENT TOOLS

1 A PDA is designed to recognize a^nb^n, but it fails for a^nb^nC^n. Critique the design and identify possible flaws in **stack operations and state transitions**. How would you improve it?

2 A student claims that **Context-Free Grammars (CFGs)**

$E \rightarrow E + T \mid T$
$T \rightarrow T * F \mid F$
$F \rightarrow (E) \mid \mathbf{id}$

can be directly converted into a Deterministic PDA (DPDA).Critically evaluate this statement with justification and suggest corrections if needed

3 A Turing Machine is designed using **multiple tracks** to solve a problem. **S**uggest alternative **programming techniques** like checking-off symbols or subroutines to do subtraction

4 While designing a PDA accepts the language

$$L = \{w \mid w \in (a + b)^* \text{ and } n_a(w) == n_b(w)\}$$

$n_a(w)$ is the number of a's in w

$n_b(w)$ is the number of b's in w, acceptance is defined only by **final state**, ignoring empty stack acceptance. Critique this design choice. Would it affect the language recognized? Explain with reasoning.

5 A system designer suggests replacing a Turing Machine with a PDA to reduce computational complexity. Critically compare **FA, PDA, and Turing Machine** capabilities and evaluate whether this replacement is valid.

SIMATS ENGINEERING
ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. CRITIQUE OF A PDA FOR a^nb^n THAT FAILS FOR $a^nb^nc^n$

The question asks to critique a PDA designed for a^nb^n that fails for $a^nb^nc^n$, identify possible flaws in stack operations and state transitions, and suggest improvements.

Aim

To identify why a PDA for a^nb^n cannot simply be extended to recognize $a^nb^nc^n$, and explain the correct improvement in terms of stack operations and states.

Why the Existing Design Can Work for a^nb^n

For a^nb^n, the PDA can push one stack symbol X for every a and pop one X for every b. When the input ends and the stack returns to the bottom marker, the counts are equal.

```
For each a: PUSH X
For each b: POP X
At end:
    if stack = Z0 -> ACCEPT
    otherwise -> REJECT
```

Possible Flaws When Extended to $a^nb^nc^n$

- The original PDA may have no state representing the c-processing phase.
- It may pop all X symbols during the b phase, leaving no information available for checking the number of c's.
- It may incorrectly allow transitions directly between phases without verifying that all b's have been matched.
- It may accept after matching a's and b's without checking the c count.
- It may use a transition that allows the wrong input order, such as a c before all b's are processed.
- It may not clearly reject extra or missing c symbols.

Important Theoretical Point

A standard single-stack PDA cannot recognize the general language $\{a^nb^nc^n \mid n \geq 1\}$, because that language is not context-free. Therefore, the correct improvement is not simply to add another state or a few more stack operations. A more powerful model, such as a Turing Machine, is required for the general $a^nb^nc^n$ language.

Improved Design for a Related PDA

If the intended language were a context-free variant such as $a^n b^n c^m$, the PDA could first match a's with b's and then independently handle the c's. But it cannot use the same single stack to independently remember n after the a-to-b matching information has already been consumed.

Comparison

Design	Capability	Result
Original PDA	Push a, pop b	Correct for $a^n b^n$
Add c state only	Tracks phase but does not preserve n	Still cannot correctly recognize general $a^n b^n c^n$
Turing Machine	Can mark and compare all three groups	Suitable for $a^n b^n c^n$

Justification

The main flaw is not merely a coding or transition mistake. The required language itself is beyond the power of a standard PDA. Therefore, adding transitions cannot solve the fundamental limitation.

Conclusion

The PDA should be retained for $a^n b^n$, but a Turing Machine should be used for general $a^n b^n c^n$. This is the strongest critique because it distinguishes an implementation error from a limitation of computational power.

2. CRITIQUE OF THE CLAIM: CFG CAN BE DIRECTLY CONVERTED TO A DPDA

The question gives the grammar E → E + T | T, T → T * F | F, F → (E) | id and asks whether it can be directly converted into a Deterministic PDA. □filecite□turn9file0□L22-L28□

Grammar

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

Evaluation of the Claim

The claim is too strong. A CFG can always be converted to an equivalent nondeterministic PDA (NPDA), but not every CFG can be directly converted to a deterministic PDA. The class of deterministic context-free languages is a proper subset of the context-free languages.

Why Direct Determinism Is Difficult

- The grammar contains alternatives such as E → E+T | T.
- At a point where E is on the stack, a deterministic machine may need to know which production to choose.
- Without suitable predictive information or grammar transformation, the PDA may need nondeterministic choices.
- Left recursion is present in E → E+T and T → T*F, which is unsuitable for straightforward predictive parsing.
- Therefore, a direct CFG-to-DPDA conversion is not guaranteed.

What Is Correct

Statement	Evaluation
Every CFG can be converted to an equivalent NPDA.	Correct.
Every CFG can be directly converted to an equivalent DPDA.	Incorrect.
Some CFGs can be transformed and recognized by a DPDA.	Correct.
The given expression grammar can be handled by deterministic parsing after suitable transformation.	Possible, using an appropriate parsing strategy.

Correction of the Grammar for Predictive Parsing

For a predictive parser, left recursion can be removed. One possible equivalent grammar is:

```
E  -> T E'
E' -> + T E' | ε

T  -> F T'
T' -> * F T' | ε

F  -> ( E ) | id
```

This grammar is better suited to LL-style predictive parsing. A deterministic parser can then make decisions using the next input symbol and parsing-table information.

Important Distinction Removing left recursion makes the grammar more suitable for deterministic parsing, but the statement 'all CFGs can be directly converted to DPDAs' is still false. Deterministic context-free language recognition has stricter requirements than general CFG recognition.

Justification

The correct conversion guarantee is CFG → NPDA. DPDA equivalence requires the language to be deterministic context-free and the grammar/parser to support deterministic decisions.

Conclusion

The student's claim is incorrect as a general statement. The grammar can be transformed for deterministic parsing, but a general CFG does not automatically have a direct DPDA equivalent.

3. ALTERNATIVES TO MULTIPLE-TRACK TURING MACHINE TECHNIQUES

The question asks for alternative programming techniques such as checking-off symbols or subroutines to do subtraction in a Turing Machine. □filecite□turn9file0□L29-L31□

Aim

To explain how subtraction can be implemented on a standard single-tape Turing Machine without relying on multiple tracks.

Technique 1 – Checking-Off Symbols

The machine can mark one symbol from the first number and match or remove one symbol from the second number. Repeated marking allows the machine to count without needing a separate track.

Example Idea

Suppose the unary input represents subtraction as 11111#111, meaning 5 − 3. The machine repeatedly marks one 1 from the first group and one 1 from the second group. After three matches, two 1s remain in the first group.

```
Initial:
11111#111

Mark one from each group:
X1111#Y11

Repeat:
XX111#YY1

Repeat:
XXX11#YYY

Second group finished.
```

```
Remaining first-group symbols = 11
Result = 2
```

Technique 2 – Subroutines

Instead of putting every operation into one huge transition table, divide the TM logic into conceptual subroutines such as FIND_START, FIND_NEXT_A, FIND_NEXT_B, MARK, RETURN_LEFT and CLEANUP.

Subroutine	Purpose
FIND_START	Move to the beginning of the working region.
FIND_NEXT_A	Find the next unmarked symbol of the first number.
FIND_NEXT_B	Find the corresponding symbol of the second number.
MARK	Change a processed symbol to a marked form.
RETURN_LEFT	Return to the beginning for the next iteration.
CLEANUP	Remove temporary markers and format the result.

Technique 3 – Delimiter Symbols

Use a delimiter such as # to separate the two numbers. The delimiter tells the machine where the first number ends and the second begins.

Technique 4 – Marking Symbols

Use different tape symbols such as X and Y to distinguish processed symbols from unprocessed symbols.

Comparison

Method	Advantage	Limitation
Multiple tracks	Easy conceptual organization	More complex machine representation
Checking-off	Works on a standard single tape	May require repeated scans
Subroutines	Simplifies machine design	Still requires detailed transitions
Delimiters	Clearly separates data	Consumes tape symbols

Justification

A standard TM has enough tape space to simulate structured computation using markers and repeated scans. Multiple tracks are a convenience for organization, not a requirement for computational power.

Conclusion

Checking-off symbols, delimiters and conceptual subroutines can replace multiple-track organization. The resulting machine may take more steps, but it remains implementable on a standard single-tape TM.

4. CRITIQUE OF FINAL-STATE-ONLY ACCEPTANCE FOR A PDA

The uploaded question contains a formatting/OCR issue in the mathematical language expression, but it clearly asks about a PDA for a language based on comparing the number of a's and b's, where acceptance is defined only by final state and empty-stack acceptance is ignored. □filecite□turn9file0□L32-L38□

Important Source Limitation

The exact mathematical expression is not readable in the extracted source, so the answer focuses on the stated design choice: final-state acceptance versus empty-stack acceptance. It does not assume an unshown language condition.

What Is Final-State Acceptance?

A PDA accepts by final state when, after the complete input is consumed, the machine is in an accepting state. The stack may still contain symbols unless the PDA's transition design explicitly requires it to be empty.

What Is Empty-Stack Acceptance?

A PDA accepts by empty stack when the complete input is consumed and the stack has been emptied according to the machine's rules. A final accepting state is not required in that acceptance mode.

Does Ignoring Empty-Stack Acceptance Change the Language?

For nondeterministic PDAs, acceptance by final state and acceptance by empty stack are equivalent in expressive power: both characterize exactly the context-free languages. Therefore, merely choosing final-state acceptance does not reduce the class of languages recognized by an NPDA, provided the PDA is designed correctly.

But There Is an Important Design Issue

- If the stack is supposed to represent unmatched symbols, accepting only because the machine enters a final state can be dangerous.
- The machine must ensure that all required input has been processed before entering the accepting state.
- Unwanted transitions should not allow the PDA to enter the final state while unmatched stack information remains.
- The acceptance condition should match the intended invariant of the stack.

Comparison

Acceptance method	Condition	Expressive power for NPDA
Final state	Input consumed + final state reached	Context-free languages
Empty stack	Input consumed + stack emptied	Context-free languages

Justification

The language recognized by a correctly constructed NPDA is not changed merely by selecting final-state acceptance instead of empty-stack acceptance. However, an incorrectly designed final-state transition can cause over-acceptance if it ignores stack information that should have been checked.

Conclusion

The design choice itself is valid for an NPDA, but the transition structure must ensure that final-state acceptance occurs only after the intended condition has been satisfied.

5. CAN A PDA REPLACE A TURING MACHINE TO REDUCE COMPLEXITY?

The question asks to critically compare FA, PDA and Turing Machine capabilities and evaluate whether replacing a Turing Machine with a PDA is valid. □filecite□turn9file0□L39-L41□

Aim

To compare the computational power of finite automata, pushdown automata and Turing Machines and determine when replacing a TM with a PDA is valid.

Model	Memory	Language capability	Typical use
FA	Finite control only	Regular languages	Lexical analysis, simple patterns
PDA	One unbounded stack	Context-free languages	Nested structures, parsing
Turing Machine	Unbounded read/write tape	Turing-recognizable/computable problems	General computation

Finite Automaton

A finite automaton has no unbounded memory. It is suitable for regular languages such as strings ending in 01 or strings containing an even number of 1s.

Pushdown Automaton

A PDA has one stack and can recognize context-free languages such as $\{a^n b^n \mid n \geq 1\}$. The stack provides unbounded memory in a restricted last-in-first-out form.

Turing Machine

A Turing Machine has an unbounded read/write tape and can move in both directions in the standard model. It can perform general algorithms and recognize languages beyond the context-free family, such as $\{a^n b^n c^n \mid n \geq 1\}$.

Power Relationship

```
Finite Automata  <  Pushdown Automata  <  Turing Machines
   Regular             Context-Free          General computation
```

Examples

Language/Problem	FA	PDA	TM
a*b*	Yes	Yes	Yes
$a^n b^n$	No	Yes	Yes
$a^n b^n c^n$	No	No	Yes
General algorithmic computation	No	No	Yes

Is Replacement Valid?

Not in general. Replacing a TM with a PDA is valid only when the required problem belongs to the computational power of a PDA, such as a context-free language recognition problem. If the application needs more general computation, multiple dependent counts, arbitrary memory access, or a language outside the context-free family, the PDA is not sufficient.

Advantages of PDA When Applicable

- Simpler than a general TM for stack-based problems.
- Natural for nested structures and context-free parsing.
- Can require less implementation complexity for the specific problem.
- May be easier to reason about formally.

Limitations

- Only one stack gives restricted memory access.

- Cannot recognize all context-free-plus languages.
- Cannot replace general-purpose computation performed by a TM.
- Replacing a TM merely for the purpose of 'reducing complexity' may sacrifice required computational power.

Justification

The correct design principle is to choose the weakest computational model that is still powerful enough for the required problem. A PDA can replace a TM only when its computational capabilities are sufficient; it cannot be used as a universal replacement.

Conclusion

FA, PDA and TM form increasing levels of computational power. A PDA is a good replacement for a TM only for problems that can be solved with stack-based context-free computation. For general TM-level problems, the replacement is invalid.

FINAL ANSWER SUMMARY

Q	Critique Topic	Main Conclusion
1	PDA a^nb^n vs $a^nb^nc^n$	Adding states cannot overcome the PDA limitation; use a TM for general $a^nb^nc^n$.
2	CFG to DPDA claim	CFG → NPDA is guaranteed; direct CFG → DPDA is not guaranteed.
3	TM subtraction	Checking-off, delimiters and subroutines can replace multiple-track organization.
4	Final-state vs empty-stack	For NPDAs, both acceptance modes have the same expressive power, but transitions must be designed correctly.
5	PDA replacing TM	Valid only when the problem is within PDA/context-free power.